

PE2 Viva Preparation Guide

Model development after feature engineering - Automobile Loan Default Prediction

IT3051 Fundamentals of Data Mining · Mini Project 2026 · Covers every model and every step after feature engineering, for the whole team. The viva can ask about any of it.

How to use this guide

- Read sections 1 to 6 once, in order. They are the story of the work.
- Section 7 has the likely questions. Say the bold answer first; add detail only if asked.
- Section 8 lists the honest answers for things that are unfinished or unverified. Know these before the viva.
- The companion file `READ.md` in the repo has the same facts with file paths.

1. The protocol every number uses

Every model in this guide is scored the same way. That is what makes the comparison fair.

- **Split:** the training data only: 95,372 rows, 8.1% default. The 20% test rows are not used for any decision.
- **Validation:** stratified, shuffled 5-fold cross-validation, `RANDOM_STATE = 42`.
- **Metrics:** ROC-AUC is the main ranking metric (threshold-free). PR-AUC is the tie-breaker. Recall, precision and F1 are at the default 0.5 threshold. Accuracy is not used, because always predicting 'no default' scores about 92%.
- **Code:** `src/evaluation/metrics.py`, function `evaluate_model`. It clones the pipeline for each fold, so no fitted state leaks between folds.

Viva line: 'Every model is scored by the same function, with the same folds, metric and threshold.'

2. The pipeline (after feature engineering)

The earlier pipeline recomputed statistics from both train and test on each call. It could not transform a single new application. It is now one fitted scikit-learn Pipeline, built by `build_pipeline(model)` in `src/preprocessing/feature_engineering.py`.

Stage	What it does	Learns from train?
Row-level cleaning (before split)	Numeric coercion, exact-duplicate removal (2,640 rows), sentinel and placeholder fixes, stratified split	No (fixed rules)
ScoreSummaryAdder	<code>score_mean</code> , <code>score_min</code> , <code>scores_available</code> , computed before imputation	No
MissingValueImputer	Median fill plus <code>_was_missing</code> flags where missing rate > 5%	Yes: medians
IncomeRatioAdder, IncomeLogTransformer	<code>credit_to_income</code> , <code>annuity_to_income</code> , <code>income_per_member</code> ; log of income	No
CategoricalOneHotEncoder	One-hot encoding of the general categorical columns	Yes: dummy columns
RareCategoryGroupier	Categories under 1% of training rows become 'Other'	Yes: rare list
CategoricalOneHotEncoder (grouped)	One-hot encoding of the 4 grouped columns	Yes: dummy columns
DerivedFeatureAdder	<code>Age_Years</code> , <code>Credit_Loan_Ratio</code> , <code>has_children</code> , <code>contact_count</code> ; drops <code>Age_Days</code> and <code>Loan_Annuity</code>	No
ImportanceFeatureSelector	Random Forest fitted on train; drops features with importance below 0.001 (58 dropped)	Yes: importances
SelectiveStandardScaler	Scales continuous columns only (more than 2 unique values)	Yes: mean and std

Stage	What it does	Learns from train?
Model (last step)	The candidate model	Yes

Verified: after transformation the training data has 95,372 rows and 46 features, with zero missing values. The top feature is score_mean (importance 0.227). A single new application can be transformed and scored by the fitted pipeline. **Tests:** tests/test_pipeline.py, 16 passing.

Viva line: 'Imputation, encoding and feature selection learn from training data only, and the model is the last step. So one fitted object can score one new application, which the backend needs.'

3. The models

Seven candidates were built. For each one, know its builder file, its settings, its cross-validated result, and one sentence on why it lands where it does.

Model	Builder	Notebook	CV ROC-AUC	Status
Logistic Regression	src/models/logistic_regression.py	05a_logistic_regression	0.734 +/- 0.005	verified
Decision Tree	src/models/decision_tree.py	decision_tree	0.708 +/- 0.004	UNVERIFIED
Random Forest (untuned)	src/models/random_forest.py	05_baseline_models_random_forest	0.774 +/- 0.004	verified
Random Forest (tuned)	same builder	07b_random_forest_tuning	0.770 +/- 0.004	verified, not kept
Gaussian Naive Bayes	src/models/gaussian_naive_bayes.py	05b_gaussian_naive_bayes	0.697 +/- 0.004	verified
XGBoost (baseline)	src/models/xgboost_model.py	05c_XGBoost_Baseline	0.763 +/- 0.004	verified
XGBoost (tuned)	same builder	07_hyperparameter_tuning	0.771 +/- 0.004	verified
LightGBM	src/models/lightgbm_model.py	lightgbm	none	PENDING

3.1 Logistic Regression

- **Settings:** L2 regularisation, C = 1.0, class_weight 'balanced', max_iter 1000, lbfgs solver. Untuned.
- **CV:** ROC-AUC 0.734, PR-AUC 0.212 +/- 0.005, recall 0.668, precision 0.153, F1 0.249.
- **Reference only (test set):** ROC-AUC 0.743. Not used for any decision.
- **Decision in its notebook:** keep as the interpretable linear baseline. Its recall at 0.5 is much higher than Random Forest's, but Random Forest ranks better overall.
- **Why it is behind:** a straight-line model cannot draw curves or interactions, so it underfits.

3.2 Decision Tree

- **Settings:** max_depth 6, min_samples_leaf 20, class_weight 'balanced', random_state 42.
- **Logged CV:** ROC-AUC 0.708 +/- 0.004, PR-AUC 0.181, recall 0.607, precision 0.152, F1 0.242.
- **Status: UNVERIFIED.** The notebook's Finding describes 5-fold CV, but no cross-validation code exists in the repo for this row. Its notebook also loads X_train.joblib files that are not in the repo.
- **Why it is behind Random Forest:** one tree with leaves of at least 20 gives coarse, step-shaped predictions. Unlimited depth overfits (training ROC-AUC near 1.0, much lower CV).
- **Advantage:** the rules can be read by a non-technical person.

3.3 Random Forest (untuned)

- **Settings:** 300 trees, class_weight 'balanced', through the shared pipeline.
- **CV:** ROC-AUC 0.774 +/- 0.004, PR-AUC 0.358 +/- 0.009, recall 0.163, precision 0.632, F1 0.260.
- **Why it works:** averaging 300 deep trees cancels their individual overfitting, so variance drops while the curves and interactions a tree can model are kept.
- **Recall at 0.5 is low:** that is the operating point, not the ranking. The threshold is a separate team decision.

3.4 Gaussian Naive Bayes

- **Settings:** default GaussianNB through the shared pipeline. No tuning (one parameter, weakest model).
- **CV:** ROC-AUC 0.697 +/- 0.004, PR-AUC 0.176 +/- 0.009, recall 0.479, precision 0.170, F1 0.250.

- **Why it is weakest:** it assumes features are independent given the class. Our features are strongly correlated (score_mean and the Score_Source columns), so the assumption breaks.
- **Why recall is higher than Random Forest's at 0.5:** its probability estimates are less conservative near the cut-off, at the cost of much lower precision.

3.5 XGBoost (baseline)

- **Settings:** 300 trees, learning rate 0.05, max depth 6, subsample 0.8, colsample 0.8, logloss. No class weighting.
- **CV:** ROC-AUC 0.763 +/- 0.004, PR-AUC 0.261 +/- 0.010, recall 0.022, precision 0.622, F1 0.042.
- **Why recall at 0.5 is so low:** without class weighting, almost no probabilities reach 0.5 for defaulters. The ranking is still good (ROC-AUC 0.763).

3.6 XGBoost (tuned)

- **Search:** RandomizedSearchCV, 30 configurations, stratified 5-fold CV, scored by ROC-AUC.
- **Best parameters:** 700 trees, learning rate 0.03, depth 8, subsample 0.7, colsample 0.8, min_child_weight 1, gamma 0.5, reg_alpha 0.01, reg_lambda 2.0.
- **CV:** ROC-AUC 0.771 +/- 0.004, PR-AUC 0.307 +/- 0.009, recall 0.045, precision 0.717, F1 0.085.
- **Compared with its baseline:** ROC-AUC up by 0.008, about twice the standard deviation, so a real gain. PR-AUC rose from 0.261 to 0.307. Recall at 0.5 is still very low.
- **Caveat:** the notebook also reports test-set results. They were not used to pick parameters, but the team has not agreed a policy on test-set reporting.

3.7 LightGBM

- **Settings:** 100 trees, learning rate 0.05, max depth 5, scale_pos_weight 11.37, random_state 42. A column-name cleaning step is added because LightGBM rejects colons and spaces in feature names.
- **Reference only (test set, from its notebook and docstring):** ROC-AUC 0.748, PR-AUC 0.215, recall 0.648, precision 0.168. Training takes about 2 seconds.
- **Status: PENDING.** It has no cross-validated result, so it cannot be compared yet. Its notebook also loads X_train.joblib files that are not in the repo.
- **Feature names need checking:** the notebook's top features (Credit_to_Annuity_Ratio, Mean_Bureau_Score, Age_Days) do not match the column names our pipeline produces. The notebook was probably run on an older feature set.

4. Tuning: what changed and what did not

Tuning means searching a grid or random set of settings and keeping the best by cross-validation. Two models were tuned, with different results.

Model	Untuned CV ROC-AUC	Tuned CV ROC-AUC	Gain	Std	Decision
Random Forest	0.774	0.770 (500 trees, depth 20, leaf 1, sqrt, balanced)	-0.004	0.004	Keep the untuned baseline. The gain is within one standard deviation.
XGBoost	0.763	0.771 (700 trees, depth 8, lr 0.03, ...)	+0.008	0.004	Tuned version is a real gain, close to Random Forest on ROC-AUC.

- **Why random search, not grid:** the same number of trials covers more values of the parameters that matter. The Decision Tree grid alone has 4,320 combinations.
- **Why the standard deviation matters:** a difference smaller than the fold-to-fold spread is noise, so it is a tie.
- **Random Forest caveat:** the exact untuned setting was not among the 30 sampled configurations. The search is not exhaustive.
- **Superseded threshold run:** an earlier Random Forest run chose an operating threshold of 0.309. It gave recall 0.650 but about 22,000 extra false alarms. The gain came from the threshold, not the tuned parameters, and the untuned baseline was never scored at that threshold. That is why the threshold was removed from the comparison.
- **Class-weight comparison (Random Forest):** balanced 0.764 > balanced_subsample 0.760 > none 0.753 (best search scores).

5. The comparison

Notebook 06b_model_comparison reads the logged CV results, keeps the most recent row per model, and does not use the test set.

Rank	Model	ROC-AUC	PR-AUC	Recall (0.5)	Precision (0.5)	Status
1	Random Forest (untuned)	0.774 +/- 0.004	0.358 +/- 0.009	0.163	0.632	verified
2	XGBoost (tuned)	0.771 +/- 0.004	0.307 +/- 0.009	0.045	0.717	verified
3	Random Forest (tuned)	0.770 +/- 0.004	0.339 +/- 0.009	0.285	0.381	verified
4	XGBoost (baseline)	0.763 +/- 0.004	0.261 +/- 0.010	0.022	0.622	verified
5	Logistic Regression	0.734 +/- 0.005	0.212 +/- 0.005	0.668	0.153	verified
6	Decision Tree	0.708 +/- 0.004	0.181 +/- 0.004	0.607	0.152	UNVERIFIED
7	Gaussian NB	0.697 +/- 0.004	0.176 +/- 0.009	0.479	0.170	verified
-	LightGBM	not in table	-	-	-	PENDING

- **Proposed shortlist (team to confirm):** Random Forest (untuned) and XGBoost (tuned). They are within one standard deviation on ROC-AUC, and Random Forest leads clearly on PR-AUC.
- **Why the shortlist is not final:** the operating threshold is not chosen, and the final model needs the team's agreement. Recall at 0.5 is very low for both top models, so the threshold matters.
- **Why LightGBM is excluded:** its only numbers are test-set results. Adding them would mix protocols.

6. Known issues (state these plainly if asked)

- **Decision Tree row unverified:** no cross-validation code exists for it in the repo, and its notebook loads files that are missing.
- **LightGBM has no CV result,** and its feature names do not match the current pipeline.
- **Test-set numbers appear in several notebooks** (Logistic Regression, Decision Tree, LightGBM, XGBoost tuning). None were used to pick anything, but the team has not agreed a rule for reporting them.
- **Duplicate rows in experiments.csv:** the Decision Tree appears twice. The comparison deduplicates on read, but the log file itself is not cleaned.
- **lightgbm is missing from requirements.txt,** so one test fails until it is added.
- **The teammate's 06_model_comparison.ipynb is broken** (a NameError, XGBoost only). We built 06b instead.
- **The operating threshold and final model are not chosen yet.** They are team decisions.
- **Backend, frontend, and the models/ output folders are still empty.**

7. Likely questions with short answers

Protocol and metrics

Q: Why do all models use the same protocol?

A: **So the comparison is fair.** Otherwise a difference could come from the split, not the model.

Q: Why cross-validation and not the test set?

A: **The test set stays unseen until the final model is chosen.** Comparisons use CV on the training data.

Q: Why not accuracy?

A: **Predicting 'no default' for everyone scores about 92%**, so accuracy does not measure anything useful here.

Q: Why ROC-AUC first and PR-AUC second?

A: **ROC-AUC measures ranking across all thresholds**, so it does not depend on the cut-off. PR-AUC is more informative when the positive class is only 8%, so it breaks ties.

Q: What does 0.774 ROC-AUC mean?

A: **For a random pair of one defaulter and one non-defaulter, the model gives the defaulter the higher risk about 77% of the time.**

Pipeline and preprocessing

Q: Why one pipeline for every model?

A: **So every model sees the same preprocessing, and one fitted object can score one new application.**

Q: Why are statistics learned only from training data?

A: **To prevent leakage.** Medians, dummy columns, rare-category lists, feature importances and scaling parameters all come from the training folds.

Q: Why is row-level cleaning done before the split?

A: **Because duplicate removal changes the row count.** Doing it before the split also stops duplicate copies landing in both train and test.

Q: Why is income log-transformed?

A: **Income is extremely right-skewed** (skew about 43 before, 0.18 after), so the log makes it more usable for linear models.

Q: Why is 'score_mean' so important?

A: **It combines the three external credit scores**, which are the strongest single predictors in the data.

Q: Why is a Random Forest used inside the feature selector too?

A: **To rank features by importance, using training data only.** It is a separate model from the final candidate, fitted inside every fold.

Models

Q: Why include a Decision Tree?

A: **It is the easiest model to explain and shows overfitting clearly**, which Random Forest then fixes by averaging.

Q: Why Logistic Regression?

A: **It is the standard credit-scoring benchmark**, and its coefficients explain direction. It is behind because it can't draw curves or interactions.

Q: Why does Random Forest beat the Decision Tree?

A: **It averages many deep trees**, which keeps the tree's curves and interactions but removes most of its variance.

Q: Why is XGBoost not the winner?

A: **On ROC-AUC it is within one standard deviation of Random Forest**, and on PR-AUC it is behind.

Q: Why Naive Bayes if it's weakest?

A: **As a contrast model**. It explains why assuming independent features fails on correlated data.

Q: Why does LightGBM not appear?

A: **It has no cross-validated result yet**. Adding test-set numbers would mix protocols.

Q: Do trees need scaling?

A: **No**. Trees split on thresholds, so scaling does not change any split. Scaling is applied only where the model needs it.

Tuning

Q: What is RandomizedSearchCV?

A: **It samples a set number of random configurations**, scores each by CV, and keeps the best.

Q: Why did Random Forest tuning not help?

A: **Its best sampled configuration was within one standard deviation of, and slightly below, the untuned baseline**. The baseline was already close to the best of the sampled space. The exact baseline was not sampled, so the search is not exhaustive.

Q: Why did XGBoost tuning help?

A: **Its ROC-AUC rose by about twice its standard deviation**. We did not test why. One possible reason is that its baseline used untuned settings, but that is a hypothesis.

Q: Why keep the untuned Random Forest?

A: **Simpler is better when the gain is not real**. The tuned version was within one standard deviation and had lower PR-AUC.

Q: Why is the threshold not part of the tuning comparison?

A: **Changing the threshold changes recall and precision without changing ranking**. Comparing a tuned threshold against an untuned one at 0.5 is unfair, so the threshold is a separate team decision.

Results and interpretation

Q: Why is recall only 0.16 for Random Forest?

A: **At the default 0.5 cut-off it is low, but that is the operating point, not the ranking**. The threshold is a team decision.

Q: Why is XGBoost's recall 0.02 at 0.5?

A: **It has no class weighting**, so almost no defaulter probabilities reach 0.5. Its ranking is still good.

Q: Why does the Decision Tree catch 61% of defaulters at 0.5?

A: **Balanced weights and a shallow tree give it more defaulter predictions**, at the cost of many false alarms (precision 0.15).

Q: What should we do about the low recall?

A: Choose the threshold deliberately, on out-of-fold predictions, with the cost of each error in mind.

Missing a defaulter and flagging a good customer have different costs.

8. Honest answers for unfinished or unverified work

- **'Is the Decision Tree number verified?'** No. Its logged CV row has no traceable cross-validation code in the repo. We will confirm its source or re-run it under the shared protocol.
- **'Why is LightGBM not in the comparison?'** It has no cross-validated result yet. Its only numbers are test-set results, which would mix protocols.
- **'Did you use the test set?'** Not for any decision. Several notebooks report test-set numbers for reference, and the team has not yet agreed a reporting rule.
- **'Is the final model chosen?'** Not yet. The shortlist is proposed. The threshold and final model are team decisions.
- **'Why is the Random Forest search not exhaustive?'** The untuned setting was not among the 30 sampled configurations. It was re-evaluated separately, so the comparison is still fair.
- **'Why does the Gaussian NB have no tuning?'** It has one parameter and is clearly the weakest model. Tuning would not change which model wins.

9. Before the viva: checklist

- Add lightgbm to requirements.txt, then run `pip install -r requirements.txt`.
- Run `pytest -q`. Expect 22 of 22 once lightgbm is installed.
- Open in this order: `src/evaluation/metrics.py`, `src/preprocessing/feature_engineering.py` (`build_pipeline`), then one notebook per model, then `06b_model_comparison.ipynb`.
- Notebooks already show outputs. Do not re-run the tuning notebooks during the viva (the Random Forest search takes 35-45 minutes).
- Know the status of the Decision Tree and LightGBM before saying anything about them.
- Be able to explain every row of the comparison table, including why LightGBM is missing.